

Završni rad - Vector Embedding i Clustering

bi li mi mogao objasniti malo o vector embeddinzima? mentor mi je rekao da ne bi bilo lose skupiti sve zadatke na hrpu i dati ai-ju da napravi embedding pa da zadatke mogu clusterirati i uociti outliere lakse. mozes li mi objasniti kako ovo napraviti?

Da. Ovo što ti je mentor rekao ima smisla, i zapravo je dosta prirodan sljedeći korak za tvoj projekt, jer već imaš zadatke vezane uz PDF-ove i već si razmišljao o izvlačenju teksta zadatka iz PDF stranice. Upravo taj tekst onda postaje input za embeddinge i kasnije klasteriranje.

Završni rad - Tick Box i Chatbot

Najjednostavnije objašnjenje:

Embedding je samo numerički vektor koji predstavlja značenje nekog teksta.

Umjesto da zadatak gledaš kao običan string, npr.:

- "Napravi DFS obilazak grafa"
- "Obiđi graf u dubinu rekursivno"
- "Prikaži redoslijed DFS obilaska"

model ih pretvori u točke u nekom višedimenzionalnom prostoru. Ako su zadaci semantički slični, njihove točke će biti blizu. Ako su jako drukčiji, bit će daleko.

To ti onda omogućuje dvije korisne stvari:

1. Clustering

Grupiraš slične zadatke zajedno.

Primjerice, jedan klaster mogu biti BST zadaci, drugi DFS/BFS, treći hash tablice, četvrti složenosti.

2. Outlier detection

Nađeš zadatke koji "strše" i ne liče previše ni na jednu grupu.

To mogu biti:

- rijetki tipovi zadataka
- loše parsirani zadaci
- zadaci koji spajaju više tema
- neki "posebni" profesorski zadaci koji odstupaju od šablone

Zašto je to korisno baš tebi

Ako radiš sustav za ASP zadatke, embeddingi ti mogu pomoći da:

- automatski otkriješ tipove zadataka bez ručnog tagiranja svega
- vidiš koji se obrasci često ponavljaju
- prepoznaš "egzotične" zadatke koje student možda treba posebno vježbati
- kasnije preporučuješ slične zadatke
- za chatbot povlačiš slične zadatke i rješenja, a ne samo isti PDF/page lookup

To se dobro naslanja na ono što već imaš, jer već radiš s tekstom zadatka iz PDF-a i idejom da se LLM-u proslijedi tekst zadatka i rješenja.

Kako to praktično napraviti

Pipeline ti je otprilike ovakav:

1. Izvuci tekst svakog zadatka

Za svaki zadatak trebaš imati barem:

- exam_type
- exam_date
- task_no
- text zadatka

Idealno i:

- page
- pdf
- eventualno solution_text ako postoji

2. Očisti tekst

Nemoj pretjerivati, ali napravi osnovno:

- ukloni višestruke razmake
- spoji slomljene retke
- makni zaglavlja tipa "JMBAG, IME I PREZIME"
- makni opće ispitne upute koje se ponavljaju na svakoj stranici

Bitno: želiš da embedding opisuje **zadatak**, a ne šum oko njega.

3. Napravi embedding za svaki zadatak

Dobiješ za svaki task jedan vektor, recimo duljine 384, 768 ili slično, ovisno o modelu.

4. Spremi embeddinge

Najjednostavnije:

- CSV + JSON nije idealno
- bolje **parquet**, **pickle**, **numpy**, ili mala baza

Za MVP može i DataFrame s jednim stupcem **embedding**.

5. Pokreni clustering

Najčešće:

- **KMeans** ako unaprijed pretpostavljaš broj grupa
- **HDBSCAN** ili **DBSCAN** ako želiš da model sam nađe guste grupe i outliere

Za tvoj slučaj bih iskreno prvo probao:

- **KMeans** za vizualni osjećaj

- DBSCAN ili HDBSCAN za outliere

6. Vizualiziraj

Pošto su embeddingi visoke dimenzije, za graf ih smanjiš na 2D:

- PCA
- t-SNE
- UMAP

Tada možeš doslovno vidjeti oblake zadataka.

Kako prepoznati outliere

Outlier je zadatak koji nema puno bliskih susjeda.

Možeš ga naći na više načina:

- **DBSCAN/HDBSCAN:** neki zadaci dobiju labelu noise
- **distance to centroid:** u KMeansu gledaš koji su najudaljeniji od centra svog klastera
- **average nearest-neighbor distance:** ako je prosječna udaljenost do 5 najbližih susjeda velika, to je sumnjiv outlier

U praksi bih ti preporučio:

- prvo napravi embeddinge
- onda **NearestNeighbors**
- pa ispiši 20 zadataka s najvećom prosječnom udaljenošću do 5 najbližih susjeda

To je vrlo jednostavno, a daje dobar prvi signal.

Na što paziti

Najveća zamka je ovo: ako embeddaš **cijelu stranicu PDF-a**, a ne samo konkretan task, dobit ćeš lošije rezultate.

Zašto? Jer će embedding biti zagađen stvarima kao:

- naslov ispita
- opće upute
- rješenja drugih zadataka
- ponavljajući tekst oko zadatka

Dakle, idealno je imati svaki zadatak izdvojen kao zaseban tekstualni zapis.

Druga zamka: embedding ne razumije savršeno strukturu koda i matematike.

Ako imaš puno C++ potpisa, pseudokoda i formula, može pomoći:

- ostaviti ih unutra, jer nose signal
- ali možda i dodati kratki ručni sažetak tipa "BST insertion task", "DFS traversal", "heap construction"

Treća zamka: klasteri neće automatski biti "školski" čisti.

Možda će jedan klaster biti "grafovi + stog/red", jer se u tekstovima često pojavljuju slični obrasci. To nije bug, nego signal kako model vidi sličnost.

Kako bih ja to napravio za tvoj projekt

Po redu:

1. Skupi sve zadatke u tablicu:

- `id`
- `exam_type`
- `exam_date`
- `task_no`
- `task_text`

2. Napravi jedan baseline embedding model.

3. Izračunaj:

- cosine similarity matricu
- nearest neighbors
- KMeans klastere
- DBSCAN/HDBSCAN outliere

4. Ručno pregledaj:

- 5 zadataka iz svakog klastera
- 10 najjačih outliera

5. Tek onda odluči:

- trebaju li ti ručni tagovi
- trebaš li spajati embedding s metapodacima tipa MI/ZI, bodovi, topic, difficulty

Vrlo je moguće da će najbolji rezultat biti **hibrid**:

- embedding od teksta zadatka
- plus metadata features kao `exam_type`, `points`, `task_no`, možda procjena težine

Minimalni Python primjer

Evo kostura kako to izgleda:

```
import pandas as pd
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import PCA

# df mora imati stupac "task_text"
df = pd.read_csv("tasks.csv")

# 1. embedding model
model = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")

# 2. izračun embeddinga
texts = df["task_text"].fillna("").tolist()
embeddings = model.encode(texts, normalize_embeddings=True)

# 3. clustering
kmeans = KMeans(n_clusters=8, random_state=42)
df["cluster"] = kmeans.fit_predict(embeddings)

# 4. nearest neighbors za outliere
nn = NearestNeighbors(n_neighbors=6, metric='cosine')
nn.fit(embeddings)
distances, indices = nn.kneighbors(embeddings)

# Prvi susjed je obično sam zadatak, pa uzmi ostale
df["avg_neighbor_distance"] = distances[:, 1:].mean(axis=1)

# Veća udaljenost = više outlier-ish
df = df.sort_values("avg_neighbor_distance", ascending=False)

print("Top outlieri:")
print(df[["task_no", "task_text", "cluster", "avg_neighbor_distance"]].head(10))

# 5. jednostavna 2D projekcija za vizualizaciju
pca = PCA(n_components=2)
coords = pca.fit_transform(embeddings)
df["x"] = coords[:, 0]
df["y"] = coords[:, 1]
```

Koji embedding model uzeti

Ako su zadaci na hrvatskom, engleski-only model nije idealan.

Gledaj nešto multilingual, npr. iz **sentence-transformers** obitelji. Za početak je sasvim dovoljno nešto tipa multilingual MiniLM. Nema potrebe da odmah koristiš neki ogroman model.

Za MVP ti ne treba savršen embedding. Treba ti dovoljno dobar embedding da:

- DFS/BFS budu blizu
- heap zadaci budu blizu
- BST zadaci budu blizu
- složenosti budu blizu
- čudni zadaci ispadnu sa strane

Kako znati je li uspjelo

Nemoj gledati samo graf i reći "izgleda okej".

Provjeri konkretno:

- Uzmi jedan zadatak o DFS-u. Jesu li mu nearest neighbors stvarno DFS/BFS/graf zadaci?
- Uzmi jedan heap zadatak. Jesu li blizu heap/sort zadaci?
- Jesu li outlieri stvarno posebni, ili su samo loše izvučeni tekstovi?

Ako nearest neighbors “imaju smisla”, na dobrom si putu.

Najbitnija intuicija

Embedding nije klasifikacija.

On ti ne kaže: “ovo je sigurno BST”.

On kaže: “ovaj zadatak je sličan ovim drugim zadacima”.

To je korisno jer:

- ne moraš unaprijed definirati sve kategorije
- možeš otkriti obrasce koje nisi ručno primijetio
- možeš graditi preporuke i retrieval na vrhu toga

Za tebe bih rekao da je najbolji prvi eksperiment:

- izdvojiti 100 do 300 zadataka
- napraviti embeddinge
- za svaki zadatak spremiti 5 najbližih susjeda
- ručno pregledati par primjera
- onda tek dodati clustering i outliere

To je dovoljno malo da ga stvarno napraviš, a dovoljno konkretno da možeš mentoru pokazati rezultat.

Ako želiš, sljedeće ti mogu napisati baš konkretan script koji uzima tvoju postojeću tablicu zadataka i radi:

1. embeddinge,
2. clustering,
3. outlier listu,
4. CSV output za pregled.

Sources

e okej. sad moje je pitanje mogu li upogoniti ai da mi ekstrahira lijepo tekst iz pdf-a. i kako se radi embedding model?

Da, možeš. I zapravo su to dvije odvojene stvari:

1. **ekstrakcija teksta iz PDF-a**
2. **pretvaranje tog teksta u embeddinge**

I važno je da ih ne pomiješaš.

1. Može li AI lijepo izvući tekst iz PDF-a?

Može, ali postoje **tri razine pristupa**.

A. Obični PDF parser, najjednostavnije

Ako je PDF "pravi" digitalni PDF, znači tekst stvarno postoji unutra, onda često ne treba AI. Dovoljno je koristiti biblioteke poput:

- `pymupdf (fitz)`
- `pdfplumber`
- `pypdf`

To je najbolje kad su ispiti normalno eksportani iz Worda, LaTeX-a ili nekog sličnog alata.

Primjer:

- parser pročitati tekst
- ti ga kasnije očistiš
- onda radiš embeddinge

Problem: PDF parser često vrati tekst ružno:

- slomljeni redovi
- čudan redoslijed stupaca
- naslov i footer pomiješani sa zadatkom
- formule i kod razbijeni

Dakle, parser je dobar za **raw extraction**, ali ne nužno za **lijepu strukturiranu ekstrakciju**.

B. OCR, ako je PDF zapravo sken

Ako je PDF samo skup slika, parser neće puno pomoći. Onda treba OCR, npr.:

- Tesseract
- EasyOCR
- Google Document AI
- Azure Document Intelligence
- neki multimodal LLM

To se koristi kad je dokument skeniran.

Problem OCR-a:

- griješi kod hrvatskih dijakritika
- zna uništiti formatiranje
- kod i matematiku često pročita loše
- ako je kvaliteta skena loša, rezultat bude kaotičan

C. LLM kao “pametni čistač” ili ekstraktor

Ovo je vjerojatno ono što ti mentor ima na umu kad kaže “daj AI-ju”.

Tu AI ne mora nužno sam čitati cijeli PDF iz nule, nego možeš napraviti pametniji pipeline:

1. izvučeš što možeš klasičnim parserom
2. pošalješ tekst ili dio stranice LLM-u
3. tražiš da:
 - izdvoji samo tekst zadatka
 - ignorira zaglavlja i potpise
 - vrati strukturirani JSON
 - prepozna gdje počinje i završava svaki zadatak

To je često najbolji pristup.

Znači ne:

- “AI magično riješi sve”

nego:

- parser/OCR izvadi sirov sadržaj
- LLM ga **normalizira i strukturira**

To ti je puno realnije i stabilnije.

Što je najbolji pristup za tvoj slučaj?

Pošto radiš sa starim ispitima i zadacima, ja bih išao ovako:

Ako je PDF digitalan

- prvo probaj `pymupdf` ili `pdfplumber`
- izvuci tekst po stranici
- onda LLM-u daj jednu stranicu i reci:
 - izdvoji sve zadatke sa stranice
 - vrati ih kao listu objekata
 - za svaki objekt vrati `task_number`, `task_text`, eventualno `points`

Ako je PDF sken

- prvo OCR
- onda isti LLM cleanup korak

Drugim riječima, **AI ti je super za strukturiranje**, ali ne mora biti prvi alat koji dira PDF.

2. Kako “upogoniti AI” da ekstrahira tekst iz PDF-a?

Najpraktičnije je razmišljati u pipelineu.

Korak 1: učitaj PDF

Za svaku stranicu:

- izvuci tekst parserom
- ako nema teksta ili je tekst smeće, koristi OCR

Korak 2: pošalji sadržaj LLM-u

Na primjer, prompt može biti tipa:

Dobit ćeš tekst jedne stranice ispita iz kolegija Algoritmi i strukture podataka.

Tvoj zadatak:

1. Izdvoji samo tekst zadatka.
2. Ignoriraj zaglavlja, upute studentima, potpise i opće informacije.
3. Prepoznaaj broj zadatka ako postoji.
4. Vрати rezultat kao JSON listu.

Format:

```
[
  {
    "task_number": "1",
    "task_text": "...",
    "points": "10"
  }
]
```

Ako radiš s multimodalnim modelom koji vidi sliku stranice, možeš mu dati i screenshot stranice, što je nekad bolje kad layout parser razbije.

3. Kako znaš treba li ti parser ili AI?

Praktično pravilo:

- ako `pymupdf` vrati dosta čist tekst, ne kompliciraj
- ako tekst izgleda raspadnuto, koristi LLM za cleanup
- ako parser vrati skoro ništa, treba OCR ili multimodalni model

4. Kako se radi embedding model?

Tu postoji bitna stvar:

u većini slučajeva ti ne treniraš svoj embedding model od nule.

To bi bilo overkill.

Najčešće radiš ovo:

1. uzmeš **već gotov embedding model**

2. daš mu tekst zadatka
3. on vrati vektor
4. te vektore koristiš za clustering, similarity, search, outliere

To je ono što ti realno treba.

Dakle, ne “radiš embedding model” od nule

Nego:

- koristiš pretrained embedding model
- eventualno ga kasnije fine-tuneaš ako baš imaš dobar razlog

5. Kako embedding model radi, intuitivno?

Recimo da imaš ova tri zadatka:

- “Implementiraj BFS obilazak grafa”
- “Napiši algoritam za obilazak grafa po širini”
- “Implementiraj AVL stablo s rotacijama”

Model prvi i drugi tekst mapira blizu jer su semantički slični. Treći će biti dalje.

Na izlazu dobiješ nešto tipa:

`[0.12, -0.44, 0.98, ...]`

taj niz brojeva je embedding.

Ti onda uspoređuješ zadatke pomoću:

- cosine similarity
- euclidean distance

6. Kako se praktično koriste embedding modeli?

Najčešći Python pristup je preko `sentence-transformers`.

Primjer:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")

texts = [
    "Implementiraj BFS obilazak grafa.",
    "Napiši algoritam za obilazak grafa po širini.",
    "Implementiraj AVL stablo."
]

embeddings = model.encode(texts, normalize_embeddings=True)

print(embeddings.shape)
```

Ovo ti vrati matricu:

- jedan red po zadatku
- više stupaca, tj. dimenzije embeddinga

Ako imaš 300 zadataka, dobit ćeš 300 vektora.

7. Kako se “radi” embedding pipeline za tvoj projekt?

Najprirodnija verzija izgleda ovako:

Faza A: ekstrakcija

Iz PDF-a napraviš tablicu:

- `source_pdf`
- `page`
- `task_number`
- `task_text`

Faza B: čišćenje

Očistiš:

- višestruke razmake
- random prijelome redaka
- ponavljajuće zaglavlje
- “IME I PREZIME”, “JMBAG”, opće upute

Faza C: embedding

Za svaki `task_text` izračunaš vektor.

Faza D: analiza

Na tim vektorima radiš:

- nearest neighbors
 - clustering
 - outlier detection
 - pretraživanje sličnih zadataka
-

8. Trebaš li trenirati vlastiti embedding model?

Skoro sigurno ne, barem ne sada.

Vlastiti embedding model treniraš tek ako:

- imaš jako puno podataka
- generički modeli rade loše
- želiš specijalizaciju baš na tvoju domenu
- znaš raditi contrastive training, positive/negative parove i evaluaciju

Za završni rad je puno razumnije:

1. uzeti gotov multilingual model
2. testirati rezultate
3. eventualno napraviti lagani fine-tuning ako baš treba

9. Što znači fine-tuning embedding modela?

To znači da već postojeći model dodatno prilagodiš svom datasetu.

Na primjer, modelu kažeš:

- ova dva zadatka su slična
- ova dva nisu slična

Pa on uči bolji prostor za tvoju domenu.

Ali za to ti treba:

- dobar skup parova
- evaluacija
- vrijeme
- GPU je poželjan

Za prvi prolaz, preskoči to.

10. Koji model uzeti?

Pošto imaš hrvatski i vjerojatno dosta tehničkog jezika, gledao bih **multilingual sentence embedding modele**.

Dobar prvi izbor je nešto iz **sentence-transformers** obitelji, primjerice multilingual MiniLM varijanta.

Zašto:

- radi solidno na više jezika
- lagan je
- jednostavan za korištenje
- dobar za baseline

11. Gdje AI stvarno pomaže, a gdje ne?

AI ti najviše pomaže u:

- prepoznavanju granica zadatka
- čišćenju ružnog teksta iz PDF-a
- strukturiranju u JSON
- semantičkom uspoređivanju zadataka preko embeddinga

AI ti ne pomaže toliko ako:

- hoćeš savršeno očuvati layout
- imaš ekstremno loše skenove
- formule i kod moraju biti 100 posto precizni bez greške

12. Minimalan realan plan za tebe

Ja bih ti preporučio ovo:

Prva verzija

1. uzmi 20 do 50 stranica PDF-ova
2. probaj `pymupdf` izvući tekst
3. za svaku stranicu pošalji tekst LLM-u da vrati:
 - broj zadatka
 - tekst zadatka
4. spremi to u CSV
5. na `task_text` izračunaj embeddinge
6. napravi nearest-neighbor pregled

To ti je već dovoljno za demo.

Druga verzija

Dodaj:

- clustering
- outlier detection
- vizualizaciju

Treća verzija

Dodaj:

- retrieval sličnih zadataka
- topic labeling klastera
- povezivanje s chatbotom

13. Jednostavan primjer ekstrakcije + embeddinga

Evo jako grub kostur:

```
import fitz
import pandas as pd
from sentence_transformers import SentenceTransformer

# 1. Učitavanje PDF-a
doc = fitz.open("ispit.pdf")

pages = []
for i, page in enumerate(doc):
    text = page.get_text()
    pages.append({
        "page": i + 1,
        "raw_text": text
    })

df = pd.DataFrame(pages)

# 2. Ovdje bi inace isao LLM cleanup
# Za sad samo glumimo da je raw_text = task_text
df["task_text"] = df["raw_text"].str.replace(r"\s+", " ", regex=True).str.strip()

# 3. Embedding model
model = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")

# 4. Racunanje embeddinga
embeddings = model.encode(df["task_text"].tolist(), normalize_embeddings=True)

print(df.head())
print(embeddings.shape)
```

Naravno, u stvarnosti ne želiš embedditi cijelu stranicu nego konkretne zadatke.

14. Najvažnija stvar koju trebaš zapamtiti

Ovo je prava mentalna slika:

- **PDF extraction** = kako doći do teksta
- **LLM cleanup/structuring** = kako tekst pretvoriti u uredne zapise zadataka
- **embedding model** = kako svaki zapis zadatka pretvoriti u brojčani vektor
- **clustering/outliers** = što radiš s tim vektorima

To su četiri odvojena sloja.

Ako ih odradiš redom, cijela stvar postane dosta jasna.

15. Što bih ja napravio na tvom mjestu

Ne bih odmah trenirao vlastiti embedding model.

Napravio bih:

- **PyMuPDF** za extraction
- LLM za čišćenje i segmentaciju zadataka
- gotov multilingual embedding model
- **scikit-learn** za clustering i outliere

To je dovoljno ozbiljno za završni rad, a nije suludo komplicirano.

U sljedećoj poruci ti mogu složiti baš konkretan end-to-end plan, s Python stackom i redoslijedom koraka, tipa: koje biblioteke instalirati, kako spremiti podatke, kako zvati model i kako dobiti prve klasterne.

okej, to sam i mislio. prije nego sto mi das precizan plan, bi li mi mogao objasniti kako radi taj embedding u smislu kako se odredi koji ce se brojevi dati, koliko dimenzija ce vektor imati itd?

Dobro pitanje. Ako razumiješ **što embedding model zapravo radi**, onda ti cijeli pipeline postane puno jasniji.

Pokušat ću objasniti od intuicije prema tehničkom.

1. Što embedding zapravo radi

Embedding model radi jednu stvar:

pretvara tekst u **vektor brojeva** koji predstavlja njegovo značenje.

Primjer:

tekst:

"Implementiraj BFS obilazak grafa"

embedding:

[0.17, -0.83, 0.41, ..., 0.09]

To može biti vektor duljine npr.:

- 384
- 512
- 768
- 1024
- 1536
- 3072

ovisno o modelu.

Ključna ideja:

slični tekstovi imaju slične vektore

pa će:

```
"BFS traversal of graph"  
"Obilazak grafa po širini"
```

biti blizu u tom prostoru.

2. Kako model zna koje brojeve staviti

To ne radi ručno.

Model je **neuronska mreža** koja je trenirana na ogromnoj količini tekstova.

U treningu model uči:

tekstovi sličnog značenja trebaju imati slične vektore

To se postiže tzv. **contrastive learningom**.

Modelu se daju parovi:

pozitivni par:

```
"A cat sits on the mat"  
"A kitten is sitting on a rug"
```

negativni par:

```
"A cat sits on the mat"  
"The stock market crashed"
```

Model uči:

- pozitivni par → vektori blizu
- negativni par → vektori daleko

Taj proces se ponavlja milijune puta.

Rezultat:

model nauči funkciju:

```
f(text) -> vector
```

3. Što se događa kad ubaciš tekst

Kad napišeš:

```
embedding = model.encode("Implementiraj BFS obilazak grafa")
```

događa se ovo:

1. tokenizacija

tekst se razbije na tokene.

Primjer:

"Implementiraj BFS obilazak grafa"

postane nešto poput:

```
["Implement", "###iraj", "BFS", "obil", "###azak", "grafa"]
```

2. tokeni se pretvaraju u vektore

svaki token ima svoj **token embedding**.

npr.

```
BFS -> [0.1, -0.7, 0.2, ...]
graf -> [0.5, 0.2, -0.4, ...]
```

to je samo lookup iz embedding matrice.

3. transformer obrada

ti token vektori prolaze kroz **transformer layers**.

to je ista arhitektura kao kod:

- BERT
- RoBERTa
- MiniLM

Svaki layer radi:

- self attention
- feed forward mreže
- normalizaciju

Time model uči **kontekst**.

Primjer:

riječ "graph" u:

```
graph theory
social graph
bar graph
```

dobit će različite reprezentacije.

4. pooling

Na kraju imaš embedding za **svaki token**.

Ali tebi treba **jedan embedding za cijelu rečenicu**.

Tu dolazi **pooling**.

Najčešće:

mean pooling

```
sentence_embedding = average(all_token_embeddings)
```

Dakle model napravi prosjek.

To postane finalni vektor.

4. Zašto je embedding višedimenzionalan

Svaka dimenzija hvata **neki latentni koncept**.

Ne znaš točno koji.

Primjeri koncepta mogu biti:

- tip problema
- domena (grafovi, stogovi, heap)
- operacija (insert, search)
- algoritamska kompleksnost
- matematička struktura

Ali model ne kaže:

```
dimenzija 52 = grafovi
dimenzija 103 = BFS
```

To su **latentne dimenzije**.

One nastaju automatski tijekom treninga.

5. Zašto 384 ili 768 dimenzija

To je **dizajnerska odluka modela**.

Transformer ima hidden size.

Primjer:

model	dimenzija
MiniLM	384
BERT base	768
BERT large	1024
OpenAI embeddings	1536

Zašto ne 20 dimenzija?

Jer bi izgubio previše informacija.

Zašto ne 10000?

Jer:

- sporije

- više memorije
- diminishing returns

U praksi se pokazalo da je 300 do 1500 dimenzija dobar kompromis.

6. Kako se mjeri sličnost

Najčešće se koristi:

cosine similarity

formula:

$$\text{cos_sim}(A,B) = (A \cdot B) / (||A|| * ||B||)$$

to mjeri **kut između vektora**.

Ako su:

- isti smjer $\rightarrow 1$
- okomiti $\rightarrow 0$
- suprotni $\rightarrow -1$

Zato često radiš:

`normalize_embeddings=True`

onda svi vektori imaju duljinu 1.

7. Kako embedding prostor izgleda

Možeš ga zamisliti kao ogroman prostor.

Primjer:

```

      heaps
      *
      *
graphs *      * sorting
      *
      dynamic programming
  
```

zadaci koji su slični tvore **oblake točaka**.

To su klasteri.

Outlieri su točke koje su daleko od svega.

8. Zašto embedding model razumije hrvatski

Multilingual modeli su trenirani na:

- Wikipediji

- paralelnim tekstovima
- prijevodima
- velikim web datasetima

Tako nauče da:

graph traversal
obilazak grafa
traversée du graphe

znače sličnu stvar.

9. Što embedding model ne razumije savršeno

Postoje slabosti.

Kod algoritama:

$O(n \log n)$
 $O(n^2)$

model to često vidi samo kao tekst.

Kod:

```
for(i=0; i<n; i++)
```

model ne razumije duboko logiku.

Ali u praksi je signal ipak dovoljno dobar.

10. Zašto embedding radi dobro za tvoj projekt

Zato što algoritamski zadaci imaju **ponavljajuće obrasce**.

Na primjer:

- "implementiraj stog"
- "push/pop operacije"
- "simuliraj stog"

svi ti tekstovi imaju sličan vokabular.

Embedding će ih prirodno grupirati.

11. Najvažnija mentalna slika

Embedding model je samo funkcija:

$f(\text{text}) \rightarrow \text{vector}$

gdje je cilj:

similar meaning $\rightarrow$ nearby vectors
different meaning $\rightarrow$ distant vectors

Ti onda radiš matematiku nad tim vektorima.

12. Mala demonstracija intuicije

Pretpostavi da model daje samo 3 dimenzije.

`[graphs, trees, sorting]`

Zadatak:

`"BFS traversal"`

embedding:

`[0.9, 0.1, 0.0]`

Zadatak:

`"AVL tree insertion"`

embedding:

`[0.2, 0.8, 0.0]`

Zadatak:

`"Merge sort"`

embedding:

`[0.1, 0.1, 0.9]`

U stvarnosti je dimenzija 384 ili 768.

Ali ideja je ista.

13. Još jedna zanimljiva stvar

Embedding prostor ima zanimljivu linearnu strukturu.

Primjer iz NLP-a:

`king - man + woman ≈ queen`

Slično se može pojaviti i u tehničkom tekstu.

14. Što je važno za tvoj rad

Za tvoj projekt nije najvažnije:

- kako se model trenira
- kako radi attention

Najvažnije je da razumiješ:

1. tekst → vektor

2. slični tekstovi → blizu
3. možeš raditi clustering
4. možeš raditi nearest neighbor search

To je dovoljno za završni rad.

Ako želiš, mogu ti još objasniti jednu stvar koja je **ključna za clustering zadataka** a skoro nitko je ne spominje:

zašto ponekad embedding klasteri budu čudni i kako ih poboljšati bez treniranja novog modela.

To je stvar koja često odluči hoće li projekt izgledati impresivno ili samo "meh".

okej a kako se određivati sto ce znaciti koja dimenzija?

Kratki odgovor: **ne određuje se ručno.**

Dimenzije u embedding su **latentne**, što znači da model sam tijekom treninga "odluči" što će koja dimenzija predstavljati.

Ali da razumiješ zašto, moramo pogledati kako model zapravo uči.

1. Nitko ne kaže modelu što dimenzije znače

Kod embeddinga **nema unaprijed definiranih dimenzija** poput:

- dimenzija 1 = grafovi
- dimenzija 2 = stabla
- dimenzija 3 = sortiranje

To bi bio ručni feature engineering, što je stariji pristup u NLP-u.

Kod modernih embeddinga dimenzije su **emergentne**. Model ih sam formira jer mu to pomaže riješiti trening zadatak.

2. Kako dimenzije nastanu tijekom treninga

Model ima ogroman broj parametara.

Jedan transformer model može imati:

- desetke milijuna parametara
- stotine milijuna
- ili više

Tijekom treninga model pokušava minimizirati **loss funkciju**.

Za embedding modele loss je često nešto poput:

- **contrastive loss**
- **triplet loss**

Primjer:

Pozitivan par:

```
"BFS traversal of graph"
"Obilazak grafa po širini"
```

Negativan par:

```
"BFS traversal of graph"
"Bubble sort algorithm"
```

Model pokušava napraviti:

```
distance(BFS, obilazak) -> mala
distance(BFS, bubble sort) -> velika
```

Kako to postiže?

Podešava **sve dimenzije embeddinga odjednom**.

3. Kako se pojavi “značenje” dimenzije

Pretpostavimo da model počne primjećivati da riječi:

```
graph
vertex
edge
BFS
DFS
adjacency
```

često dolaze zajedno.

Tada će neke dimenzije početi reagirati na taj uzorak.

Primjer:

```
dim_147 -> graf koncept
```

Ali to nije eksplicitno zapisano. To je samo **statistički efekt treninga**.

Druga dimenzija može reagirati na nešto drugo:

```
dim_203 -> sorting koncept
```

Treća možda reagira na:

```
dim_91 -> matematička notacija
```

Ali nijedna dimenzija nije čista.

Svaka dimenzija obično nosi **kombinaciju više signala**.

4. Važna stvar: značenje nije u jednoj dimenziji

Velika zablude je misliti da:

```
dim 50 = grafovi
dim 72 = stabla
```

U stvarnosti značenje je **raspršeno preko više dimenzija**.

Primjer:

graf zadaci mogu imati signal u dimenzijama:

```
12, 37, 91, 144, 280
```

Dok heap zadaci imaju signal u:

```
7, 41, 99, 201
```

Dakle značenje je **distribuirano**.

To se zove:

distributed representation

5. Zašto model koristi više dimenzija

Jer je to fleksibilnije.

Ako bi svaka dimenzija bila jedan koncept, model bi bio ograničen.

Distribuirane reprezentacije omogućuju modelu da hvata:

- semantiku
- sintaksu
- kontekst
- strukturu rečenice
- domenu teksta

sve u isto vrijeme.

6. Primjer iz stvarnog embedding prostora

Recimo da model vidi ova tri teksta:

```
Implementiraj BFS obilazak grafa
Implementiraj DFS obilazak grafa
Implementiraj AVL stablo
```

embeddingi bi mogli izgledati ovako (pojednostavljeno):

```
BFS -> [0.72, -0.14, 0.55, 0.01, ...]
DFS -> [0.69, -0.10, 0.58, 0.02, ...]
AVL -> [0.10, 0.77, -0.30, 0.05, ...]
```

BFS i DFS su slični jer:

- više dimenzija ima slične vrijednosti

AVL je drugačiji jer:

- druga kombinacija dimenzija aktivna.

7. Kako model odluči koliko dimenzija koristiti

To određuje **arhitektura modela**.

Transformer ima parametar:

`hidden_size`

Primjeri:

model	dimenzija
MiniLM	384
BERT base	768
BERT large	1024
OpenAI text embedding	1536

To znači:

svaki token i svaka rečenica se predstavlja vektorom te duljine.

To je dizajnerski kompromis između:

- kapaciteta modela
- brzine
- memorije

8. Zašto model ne koristi manje dimenzija

Zamisli da imaš samo 3 dimenzije:

`[x, y, z]`

To je premalo da predstaviš kompleksno značenje jezika.

Sa 384 dimenzije model može razdvojiti:

- teme
- stil
- sintaksu
- domenu
- kontekst

9. Može li se vidjeti što dimenzija znači?

Teoretski možeš pokušati.

Jedna metoda je:

pogledaš tekstove koji imaju **najveću vrijednost u toj dimenziji**.

Na primjer:

`dim_212 highest texts:`

`graph traversal
vertex adjacency
BFS algorithm`

Onda zaključiš:

`dim_212 ≈ graf signal`

Ali to je samo aproksimacija.

10. Bitna stvar za tvoj projekt

Za clustering zadataka **nije bitno što znači svaka dimenzija**.

Bitno je samo:

`distance(vector_a, vector_b)`

Ako su zadaci slični, embedding će biti blizu.

To je sve što trebaš.

11. Dobra analogija

Zamisli da opisuješ osobu sa 300 karakteristika:

- visina
- težina
- IQ
- boja kose
- brzina trčanja
- itd.

Embedding radi slično, ali:

- karakteristike nisu eksplicitne
- model ih sam otkrije
- ima ih stotine

12. Zašto je ovo zapravo moćno

Zato što model može otkriti **skrivenе strukture** u podacima.

Na primjer, mogao bi otkriti da zadaci o:

heap
priority queue
dijkstra

imaju neku latentnu povezanost.

Iako ih nikad nisi ručno označio.

13. Najvažnija mentalna slika

Embedding prostor je samo:

N-dimenzionalni prostor

u kojem:

značenje = pozicija

Zadaci su **točke u tom prostoru**.

Clustering samo traži **oblake točaka**.

Ako želiš, mogu ti objasniti još jednu stvar koja je jako korisna za tvoj projekt:

zašto embeddingi često automatski grupiraju algoritamske zadatke po temama (grafovi, heap, stabla) čak i kad model nikad nije treniran na ASP zadacima.

To je dosta zanimljiv fenomen i zapravo je razlog zašto će ti ovaj pristup vrlo vjerojatno raditi.

imam jedno pitanje vezano uz ove embeddinge. sjecam se da su neki znanstvenici u claudu nasli neke "neurone" (ne znam jesu li točno to) u Claudeu koji se asociraju sa Golden Gate Bridgeom. Kad su ih pojacali, Claude je poceo govoriti da je on Golden Gate Bridge i stalno ga je spominjao u razgovorima. Moze li se povuci paralela i s ovime sto si ti sad spomenuo? Zato nam je u biti i tesko shvatiti odakle LLM-ovima njihova izranjajuca svojstva kad je tesko pinpointati konkretno sto pojedini neuron reprezentira.

Da, postoji **vrlo direktna paralela** između onoga što si spomenuo o "Golden Gate Bridge neuronu" i onoga kako rade embedding dimenzije i općenito reprezentacije u LLM-ovima. Zapravo, to je dio šireg područja koje se zove **mechanistic interpretability** u Machine Learning.

Ali treba razjasniti jednu stvar: u tim primjerima obično **nije jedan neuron = jedan koncept**. Ono što su istraživači našli je nešto malo suptilnije.

1. Što su istraživači zapravo našli

U nekim modelima su našli **neurone ili male skupine neurona** koje se snažno aktiviraju kada se pojavi određeni koncept.

Primjer:

- neuron koji reagira na **Golden Gate Bridge**
- neuron koji reagira na **Python code**
- neuron koji reagira na **tabular data**

Kad umjetno **pojačaš aktivaciju tog neurona**, model počne:

- više spominjati taj koncept
- vidjeti ga gdje ga nema
- čak tvrditi da je on taj koncept

To je upravo ono što si opisao.

To se događa jer taj neuron nosi **dio semantičkog signala** za taj koncept.

2. Kako je to povezano s embedding dimenzijama

Embedding dimenzije su zapravo **vrlo slična stvar**.

Svaka dimenzija embedding vektora je analogna:

- jednoj “osi” u prostoru
- ili vrlo grubo jednom neuronu u posljednjem sloju

Primjer:

embedding:

```
[0.13, -0.52, 0.91, ...]
```

svaka komponenta:

```
dim_1
dim_2
dim_3
...
```

je rezultat aktivacije nekog neuronskog puta u mreži.

Dakle:

embedding dimenzije = **projekcija aktivnosti neuronske mreže**

3. Zašto ponekad vidimo “koncept neurone”

To je emergentni efekt treninga.

Ako model često vidi tekstove o:

- Golden Gate Bridge
- San Francisco
- suspension bridges
- California

onda se tijekom treninga može pojaviti neuron koji reagira na taj **statistički uzorak**.

To nije programirano.

Model jednostavno otkrije da je korisno imati takav signal.

4. Zašto je to rijetko “čist neuron”

U modernim modelima koncepti su uglavnom **distribuirani**.

To znači:

umjesto

neuron 145 = Golden Gate Bridge

češće imamo

kombinacija 30 neurona ≈ Golden Gate Bridge

To se zove **distributed representation**.

Zato je teško točno reći:

ovaj neuron znači X

5. Zašto ponekad izgleda kao da postoji “pojam neuron”

To se događa jer:

neki neuroni postanu **dominantni nositelji signala**.

Drugim riječima:

iako koncept koristi više neurona, jedan od njih nosi najveći dio signala.

Pa kad ga pojačaš, model počne “halucimirati” taj koncept.

6. Kako to dovodi do emergentnih svojstava

Emergentna svojstva LLM-ova dolaze iz toga što mreža ima:

- milijarde parametara
- tisuće dimenzija reprezentacije
- kompleksne interakcije između slojeva

Kombinacije tih faktora stvaraju:

- apstraktne pojmove
- semantičke odnose
- logičke strukture

I sve to **bez eksplicitnog programiranja**.

7. Zašto je teško “pinpointati” značenje neurona

Tri glavna razloga.

1. Distribuirane reprezentacije

Koncept je raspodijeljen preko više neurona.

2. Superpozicija

Jedan neuron može predstavljati **više koncepata odjednom**.

Primjer:

neuron može reagirati na:

- mostove
- San Francisco
- turističke atrakcije

jer se ti pojmovi često pojavljuju zajedno u podacima.

3. Interakcije slojeva

Značenje se često formira **tek kroz više slojeva transformera**.

Neuron u jednom sloju može biti samo dio šireg uzorka.

8. Kako se to odnosi na embeddinge

Embedding koji dobiješ je zapravo:

komprimirana reprezentacija svih tih neuronskih aktivacija.

Dakle:

```
tekst
↓
transformer slojevi
↓
neuronske aktivacije
↓
pooling
↓
embedding vektor
```

Svaka dimenzija embeddinga je kombinacija mnogih neuronskih signala.

9. Zato je embedding prostor zanimljiv

U embedding prostoru često vidiš:

- semantičke klastere
- linearne odnose
- analogije

Primjer iz NLP-a:

`king - man + woman ≈ queen`

To je dokaz da model uči **apstraktne strukture**.

10. Paralela s tvojim projektom

Kod zadatka iz algoritama možeš dobiti latentne osi poput:

- graf algoritmi
- stabla
- sortiranje
- dinamičko programiranje
- složenost

Ali nijedna dimenzija neće biti čisto "graf".

To će biti kombinacija više dimenzija.

Zato clustering i radi.

11. Zašto je ovo još uvijek aktivno istraživanje

Čak ni istraživači koji rade na modelima poput Claude ili GPT-4 ne razumiju u potpunosti:

- kako točno nastaju koncepti
- kako se znanje organizira u mreži
- zašto se neka emergentna svojstva pojave

To je jedan od velikih otvorenih problema u Artificial Intelligence.

12. Najbliža analogija iz ljudskog mozga

Zanimljivo je da je slično i u neuroznanosti.

Postoje tzv.:

- **grandmother neurons**

neuroni koji reagiraju na određeno lice ili objekt.

Ali čak i u mozgu većina reprezentacija je **distribuirana**, baš kao u LLM-ovima.

13. Jedna važna stvar koju većina ljudi ne shvati

Embedding prostor nije samo slučajan.

On je **visoko strukturiran semantički prostor**.

To znači da model uči neku vrstu "karte značenja".

Zato možeš:

- raditi clustering
- tražiti slične tekstove
- graditi recommendation sustave

Ako želiš, mogu ti pokazati još jednu jako zanimljivu stvar koja će ti zapravo pomoći za završni rad:

zašto embedding modeli često spontano grupiraju zadatke po algoritamskim temama (grafovi, stabla, heap) čak i ako nikad nisu trenirani na informatičkim ispitima.

Razlog je dosta zanimljiv i povezan je s načinom na koji modeli organiziraju tehnički jezik.

okej dosta objasnjava, daj mi actionable plan za sutra da odradim ovo ekstrahiranje iz pdf-a, embeddanje i skupljanje potencijalnih outliera.

Super, idemo **konkretno i minimalno**, da ovo možeš stvarno napraviti **u jednom danu**. Cilj je imati:

1. tekst zadataka iz PDF-ova
2. embedding za svaki zadatak
3. listu potencijalnih outliera

Ništa fancy. Samo pipeline koji radi.

Plan za sutra (4 faze)

1. **Extract tekst iz PDF-ova**
2. **Napraviti dataset zadataka**
3. **Generirati embeddinge**
4. **Naći outliere**

Ako imaš 200–500 zadataka, sve će raditi vrlo brzo.

0. Instalacija (10 min)

Napravi novi Python environment i instaliraj:


```
pip install pymupdf pandas sentence-transformers scikit-learn tqdm
```

Biblioteke:

- `pymupdf` → čitanje PDF-a
- `pandas` → dataset
- `sentence-transformers` → embedding
- `scikit-learn` → nearest neighbors
- `tqdm` → progress bar

1. Ekstrakcija teksta iz PDF-a

Napravimo skriptu:

`extract_tasks.py`

```
import fitz
import pandas as pd
from pathlib import Path

pdf_folder = "pdfs"

rows = []

for pdf_path in Path(pdf_folder).glob("*.pdf"):
    doc = fitz.open(pdf_path)

    for page_index, page in enumerate(doc):
        text = page.get_text()

        rows.append({
            "pdf": pdf_path.name,
            "page": page_index + 1,
            "text": text
        })

df = pd.DataFrame(rows)

df.to_csv("raw_pages.csv", index=False)

print("Saved raw_pages.csv")
```

Folder struktura:

```
project
  pdfs/
    ispiti1.pdf
    ispiti2.pdf
  extract_tasks.py
```

Run:

```
python extract_tasks.py
```

Dobiješ:

```
raw_pages.csv
```

2. Čišćenje teksta (minimalno)

Napravimo drugi script:

`clean_tasks.py`

```
import pandas as pd
import re

df = pd.read_csv('raw_pages.csv')

def clean_text(text):
    if not isinstance(text, str):
        return ""

    text = text.replace('\n', " ")
    text = re.sub(r"\s+", " ", text)
    return text.strip()

df["task_text"] = df["text"].apply(clean_text)
df = df[df["task_text"].str.len() > 50]
df.to_csv('tasks_clean.csv', index=False)
print("Saved tasks_clean.csv")
```

Run:

`python clean_tasks.py`

Sad imaš dataset.

3. Embedding

Napravimo:

`embeddings.py`

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from tqdm import tqdm

df = pd.read_csv('tasks_clean.csv')

model = SentenceTransformer(
    "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
)

texts = df["task_text"].tolist()

embeddings = model.encode(
    texts,
    normalize_embeddings=True,
    show_progress_bar=True
)

import numpy as np

np.save('task_embeddings.npy', embeddings)

df.to_csv('tasks_with_text.csv', index=False)
print("Saved embeddings")
```

Run:

`python embeddings.py`

Dobiješ:

```
task_embeddings.npy  
tasks_with_text.csv
```

4. Outlier detection

Napravimo:

find_outliers.py

```
import numpy as np  
import pandas as pd  
from sklearn.neighbors import NearestNeighbors  
  
df = pd.read_csv("tasks_with_text.csv")  
embeddings = np.load("task_embeddings.npy")  
  
nn = NearestNeighbors(  
    n_neighbors=6,  
    metric='cosine'  
)  
  
nn.fit(embeddings)  
  
distances, indices = nn.kneighbors(embeddings)  
  
avg_distance = distances[:,1:].mean(axis=1)  
  
df["avg_neighbor_distance"] = avg_distance  
  
df = df.sort_values(  
    "avg_neighbor_distance",  
    ascending=False  
)  
  
df.to_csv("outliers_ranked.csv", index=False)  
  
print(df.head(20)[["pdf", "page", "avg_neighbor_distance"]])
```

Run:

```
python find_outliers.py
```

Dobiješ:

```
outliers_ranked.csv
```

Top redovi su potencijalni outlieri.

5. Kako provjeriti radi li

Otvorite outliers_ranked.csv.

Pogledaj prvih 20 zadataka.

Traži:

- čudne zadatke
- loše parsiranje
- drugačije teme
- zadatke koji nisu ASP

To su outliers.

6. Bonus (ako imaš vremena)

Dodaj **clustering**.

Napravimo `clusters.py`:

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans

df = pd.read_csv("tasks_with_text.csv")
embeddings = np.load("task_embeddings.npy")

kmeans = KMeans(
    n_clusters=8,
    random_state=0
)

labels = kmeans.fit_predict(embeddings)

df["cluster"] = labels

df.to_csv("task_clusters.csv", index=False)
```

Sad možeš vidjeti:

```
cluster 0 → grafovi
cluster 1 → stabla
cluster 2 → sortiranje
```

Realno vrijeme

Ako imaš ~300 zadataka:

korak	vrijeme
PDF extraction	2 min
cleaning	1 min
embeddings	1-2 min
outliers	<1 min

Sve zajedno: **~10 minuta runtime**.

Minimalni rezultat koji možeš pokazati mentoru

Ako sutra napraviš ovo, već imaš:

- dataset zadataka
- embeddinge
- klastere
- outlier listu

I možeš reći:

"Napravio sam embedding reprezentaciju svih zadataka i identificirao outliere na temelju semantičke udaljenosti."

To već zvuči vrlo ozbiljno.

Ako želiš, mogu ti još dati **3 mala poboljšanja (15 linija koda)** koja drastično poboljšaju kvalitetu outliera i klastera. To su stvari koje većina ljudi zaboravi napraviti kod embedding projekata.

okej, a jesi bacio oko kako moji pdf-ovi ispita izgledaju. pogledaj to. i baci oko na pdf-ove da znas kako mi projekt zasad izgleda i sto sam dosad napravio pa vidi jesi li dobro napisao programe. ne kazem da su krivi, nego samo zelim da imas dobar kontekst i razumijevanje prije nego sto mi das kod.

Da, sad sam bacio oko i na ispitne PDF-ove i na tvoje projektne PDF-ove, i to dosta mijenja što bih ti preporučio.

Glavna stvar: **moj prethodni plan je bio dobar kao generički baseline, ali za tvoje konkretne PDF-ove ga treba prilagoditi.**

Što sam vidio u tvojim PDF-ovima

Ispitni PDF-ovi, MI i ZI

Izgledaju dosta uredno i konzistentno:

- na vrhu su uvijek zaglavlja tipa **JMBAG, IME I PREZIME**, naslov ispita i datum
- zadaci su jasno označeni kao **Zadatak 1.**, **Zadatak 2.** itd.
- svaki zadatak često ima i bodove u naslovu
- unutar zadatka imaš:
 - običan opis
 - kod blokove
 - ponekad slike, stabla, grafove, tablice
 - ponekad dio zadatka **a)** i **b)**
- **zadaci se mogu prelijevati na sljedeću stranicu**
- jedna stranica može sadržavati:
 - kraj jednog zadatka
 - početak drugog
 - više zadataka odjednom

To je bitno jer znači da moj raniji pristup:

- jedna stranica = jedan zapis

nije dovoljno dobar za tvoj slučaj.

Kod tebe treba ići:

- **detekcija granica zadatka po markeru Zadatak X.**
- pa spajanje teksta dok ne dođe sljedeći Zadatak Y.

Dakle, **jedinica za embedding ne smije biti stranica**, nego **pojedini zadatak**.

Projektni PDF-ovi

Oni nisu klasična dokumentacija projekta, nego uglavnom exportovi prethodnih razgovora i opisa ideja.

Iz njih se vidi da je tvoj projekt zasad otprilike ovo:

- Streamlit MVP
- koristiš CSV indeks zadataka
- imaš scoring i prioritizaciju zadataka
- želiš filtre tipa MI / ZI
- želiš chatbot ili barem RAG-like tutor koji ne izmišlja, nego vuče stvarne zadatke i citira izvor
- već razmišljaš o optimizaciji, LLM-ovima i sljedećim koracima

To je korisno jer znači da embedding pipeline ne radiš izolirano, nego kao potencijalni dio šireg sustava:

- preporuka zadataka
- grupiranje po tipovima
- outlier detekcija
- kasnije retrieval za chatbot

Zaključak nakon pregleda

Da budem direktan:

prethodni kod koji sam ti dao je pregrub za tvoje PDF-ove.

Ne zato što je potpuno kriv, nego zato što:

1. radi ekstrakciju po stranici, a ne po zadatku
2. ne uzima u obzir da se zadatak može protezati preko više stranica
3. ne odvaja dobro:
 - zaglavlje ispita
 - opće upute
 - sadržaj samog zadatka
4. ne čuva dovoljno metadata za kasniju provjeru

Za tvoj projekt bi bolji pristup bio ovaj:

Pravi actionable plan za tvoj konkretan slučaj

Faza 1, ekstrakcija po stranicama, ali samo kao međukorak

Prvo i dalje izvučeš tekst po stranici iz PDF-a, ali to nije finalni dataset.

Za svaki page spremiš:

- `pdf_name`
- `page_number`
- `raw_text`

To je samo sirovina.

Faza 2, segmentacija na zadatke

Ovo je ključ.

Trebaš iz raw teksta prepoznati pojave tipa:

- **Zadatak 1.**
- **Zadatak 2.**
- **Zadatak 3.**

i onda graditi task dok ne dođe sljedeći marker.

To znači:

- ako zadatak krene na stranici 1 i nastavi se na stranici 2, ti ga spojiš u jedan zapis
- ako su na istoj stranici dva zadatka, razdvojiš ih

Za svaki task zapis spremi:

- `exam_type` = MI ili ZI
- `exam_date` ako ga uspiješ izvući
- `task_number`
- `start_page`
- `end_page`
- `task_text`

To je dataset za embedding.

Faza 3, cleaning, ali pažljivo

Kod tebe cleaning ne smije biti preagresivan.

Trebaš ukloniti:

- JMBAG
- IME I PREZIME
- opće ispitne upute na vrhu
- višestruke razmake
- suvišne prijelome redaka

Ali trebaš zadržati:

- kod
- opise algoritama
- nazive struktura podataka
- a) i b) podzadatke
- bitne konstante i prototipe funkcija

Drugim riječima, želiš očistiti šum, a ne sterilizirati zadatak.

Faza 4, embedding

Tek nakon toga ide embedding nad `task_text`.

Za tebe je smislen baseline:

- `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

To je dovoljno dobro za prvi prolaz.

Faza 5, outlieri

Nemoj prvo ići na fancy anomaly detection.

Za prvi prolaz radi:

- `NearestNeighbors`
- cosine distance
- prosjek udaljenosti do 5 najbližih susjeda

Oni s najvećom udaljenošću su kandidati za outlieri.

To ti je dovoljno jednostavno i obranjivo.

Što bih sad promijenio u kodu

Najveća promjena je ova:

Umjesto:

- `extract_pages -> clean_pages -> embed_pages`

trebaš:

- `extract_pages -> segment_tasks -> clean_tasks -> embed_tasks -> rank_outliers`

To je prava verzija za tvoje ispite.

Moj savjet za sutra, praktično

Nemoj pokušavati odmah raditi:

- OCR
- vizualnu analizu slika zadatka
- multimodalni parsing
- savršeno prepoznavanje svih tablica i dijagrama

Za sutra ti je dovoljno:

1. izvući tekst iz MI i ZI PDF-ova
2. regexom segmentirati po **Zadatak X**.
3. ručno pregledati prvih 20 zapisa da vidiš jesu li granice dobre
4. očistiti tekst
5. napraviti embeddinge
6. izbaciti top 20 outliera
7. ručno pogledati jesu li stvarno outlieri ili samo loše segmentirani zadaci

To je realan i dobar dan rada.

Što očekujem da će biti problematično

Na temelju izgleda tvojih ispita, najviše problema ćeš imati s:

- zadacima koji počnu na jednoj i završe na drugoj stranici
- zadacima koji imaju puno koda i tablica
- zadacima gdje je dio sadržaja u slici ili dijagramu
- malim "box" dijelovima poput **Rješenje:** koji mogu završiti u tekstu

Ali za embedding i outliere to nije katastrofa.

Čak i ako ekstrakcija nije savršena, često će i dalje biti dovoljno dobra da prepoznaš teme i čudne zadatke.

Ukratko, moja revidirana procjena

Da, sad kad sam pogledao tvoje materijale:

- **opći smjer koji sam ti dao je dobar**
- **konkretna implementacija mora biti task-based, a ne page-based**
- tvoj projekt je već dovoljno definiran da embeddingi imaju smisla
- ovo se dobro uklapa u tvoj Streamlit MVP kao sljedeći ozbiljan korak

Sljedeće što bih ti dao je baš **novi kod prilagođen tvojim PDF-ovima**, dakle:

- ekstrakcija po stranici
- segmentacija po **Zadatak X**.
- cleaning
- embedding
- outlier ranking

i to u 3 do 4 skripte koje možeš odmah vrtjeti.